Introduction to Quantum Computing
Second Lab Exercise: QUIRK & QISKIT

February 2024

1 Introduction

On this lab practise, we will be using both `QUIRK` and `IBM QUANTUM LAB / QISKIT` services. For `QUIRK`, you can access the browser-based simulator on

- `https://algassert.com/quirk`

For `IBM QUANTUM LAB`, you need to create a free account on

- `https://lab.quantum.ibm.com/`

in case you did not create it on the last session.

For `QUIRK` usage, you can refer to the previous lab script. For `IBM QUANTUM LAB / QISKIT`, you can refer to the official documentation on:

- `https://docs.quantum.ibm.com/`

and to the `BasicExample` that we attach to this lab script. In that regard, the `transpiler` purpose is turning your quantum circuit into a simpler circuit that is suitable for either a certain quantum simulator (as it will be the case of this practice) or either an actual quantum computer. In this simple example, you can use whatever `optimization_level` you want. On an actual computation, you should consider whether the increased transpiling time is worth the more optimized circuit you obtain.

The `shots` argument on `backend.run` function fixes the number of shots of the quantum circuit. That is, the number of events on the histogram of measurements. Of course, the execution time will grow linearly on the number of shots. But the statistical significance of your analysis will grow as well.

The `job_tags` argument on the same `backend.run` function has no actual effect on your computation. It is just an array of strings that will be used for tagging your jobs on the control panel[1] of your `IBM QUANTUM LAB` service. Although you can live without these tags if you stick to free quantum simulation time, the usage of tags for accountability purposes turns critical when you switch to paid computer time on actual quantum computers. They are also critical in case your jobs have to queue for several hours. This happens if you use your free 10 min allowance on a selection of IBM quantum computers. In such a case, it is very possible that your python kernel gets killed while queuing and you have to rely on your tags to recover the result of your precious quantum computer time. On top of using tags, it would be very advisable to print your job id before making your python kernel to sleep until it can retrieve your results by means of the `job.result()` instruction. In any case, tags are highly advisable. Hence, you should start using meaningful tags for your jobs.

Finally, we require you to perform two statistical tests: the binomial test, for the two possible measurements of a single qubit, and the χ^2 one, for the histogram of more than two possible final measurements. You can refer to the following `scipy` (Python) documentation:

- Normal distribution test:
  `https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.binomtest.html`
- χ^2 test:
  `https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chisquare.html`

[1] Accessible on `https://quantum.ibm.com/jobs`

In these two pieces of documentation you have useful references for the mathematical nature of both statistical tests. Furthermore, if you followed a subject on statistics, you should be already familiar with both statistical tests. In any case, a *low* p-value marks a result whose difference from the expected theoretical value is statistically significant. A p-value below 0.05 or 0.01 should catch your attention in this regard.

2 Entanglement and Superdense Coding

Using a Hadamard gate and a `CNOT`[2] gate whose control is applied to the output of the Hadamard gate, build a quantum circuit on `QISKIT` that turns a 2-qubit initial state $|0\rangle \otimes |0\rangle$ to the following Bell states:

1. $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle)$
2. $|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |0\rangle - |1\rangle \otimes |1\rangle)$
3. $|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |1\rangle + |1\rangle \otimes |0\rangle)$
4. $|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |1\rangle - |1\rangle \otimes |0\rangle)$

Answer the following questions:

1. Simulate each state 1000 times on `QISKIT` and measure both qubits.
2. Use a χ^2 test to check that your measurements are correct once quantum fluctuations are taken into account. You can use the function `chisquare` from the Python package `scipy.stats`.
3. Using superdense coding techniques, check that you generate the correct states.

3 Quantum teleportation

Follow these steps:

1. Implement the quantum teleportation algorithm on `QUIRK`. Note that you will need the so-called *measurement gate*. Note that a quantum gate can be controlled by a (classical) bit resulting from the measurement of a quantum qubit.
2. Check on `QUIRK` that you can teleport the states $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$, $|i\rangle = (|0\rangle + j\,|1\rangle)/\sqrt{2}$ and $|-i\rangle = (|0\rangle - j\,|1\rangle)/\sqrt{2}$.
3. Implement the quantum teleportation algorithm on `QISKIT`. Note that the quantum measurement will project the qubit itself into either $|0\rangle$ or $|1\rangle$ depending on the result of the measurement, on top of its output as a classical bit. In this way, we can use the output of a quantum measurement as the control of a quantum gate. A classical bit can be also used as control for a quantum gate.
4. Teleport the quantum states $|0\rangle$, $|1\rangle$, $|+\rangle$ and $|-\rangle$. Generate 1000 measurements of the teleported qubit. Check that you measure the correct qubit once quantum fluctuations are taken into account. Note that for $|0\rangle$ and $|1\rangle$ you should exactly measure 0 and 1, respectively. The statistical analysis will be needed for $|+\rangle$ and $|-\rangle$ states. For that, you can use a binomial statistical test by means of the function `binomtest` from the Python package `scipy.stats`.
5. Apply a Hadamard gate to your teleported qubit. Repeat your measurements and your statistical analysis. However, this time $H\,|+\rangle = |0\rangle$ and $H\,|-\rangle = |1\rangle$.

[2] `NOT` gate with control.

BasicExample

March 14, 2024

[1]:
```
# Importing standard Qiskit libraries
from qiskit import QuantumCircuit, transpile, QuantumRegister, ClassicalRegister
from qiskit.visualization import *
from ibm_quantum_widgets import *

# qiskit-ibmq-provider has been deprecated.
# Please see the Migration Guides in https://ibm.biz/provider_migration_guide␣
 ↪for more detail.
from qiskit_ibm_runtime import QiskitRuntimeService, Sampler, Estimator,␣
 ↪Session, Options

# Loading your IBM Quantum account(s)
service = QiskitRuntimeService(channel="ibm_quantum")

# Invoke a primitive. For more details see https://docs.quantum.ibm.com/run/
 ↪primitives
# result = Sampler().run(circuits).result()
```

```
qiskit_runtime_service.__init__:INFO:2024-03-14 11:43:04,552: Default instance:
ibm-q/open/main
```

[2]:
```
qa = QuantumRegister(2,"reg_a")
qb = QuantumRegister(2,"reg_b")
ca = ClassicalRegister(2,"clas_a")
cb = ClassicalRegister(2,"clas_b")

circ = QuantumCircuit(qa, qb, ca, cb)
```

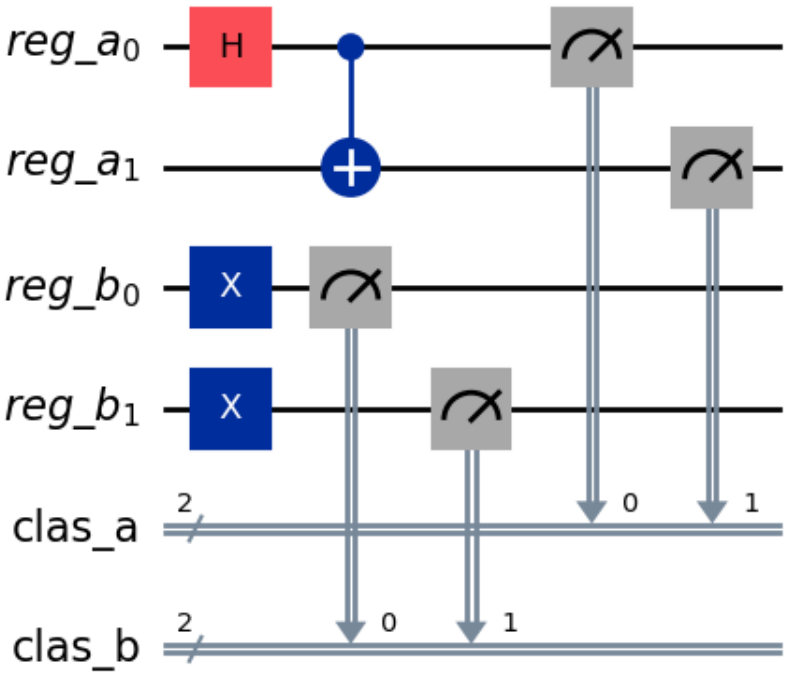
[3]:
```
circ.h(qa[0])
circ.cx(qa[0],qa[1])

circ.x(qb)

circ.measure(qa,ca)
circ.measure(qb[0],cb[0])
circ.measure(qb[1],cb[1])
```

[3]:
```
<qiskit.circuit.instructionset.InstructionSet at 0x7f282fb3d4b0>
```

[4]:
```
backend = service.get_backend("ibmq_qasm_simulator")
tr_circ3 = transpile(circ, backend, optimization_level=3) #optimization_level=1␣
↪for faster; optimization_level=0, do not optimize
tr_circ1 = transpile(circ, backend, optimization_level=1) #optimization_level=1␣
↪for faster; optimization_level=0, do not optimize
tr_circ0 = transpile(circ, backend, optimization_level=0) #optimization_level=1␣
↪for faster; optimization_level=0, do not optimize
```

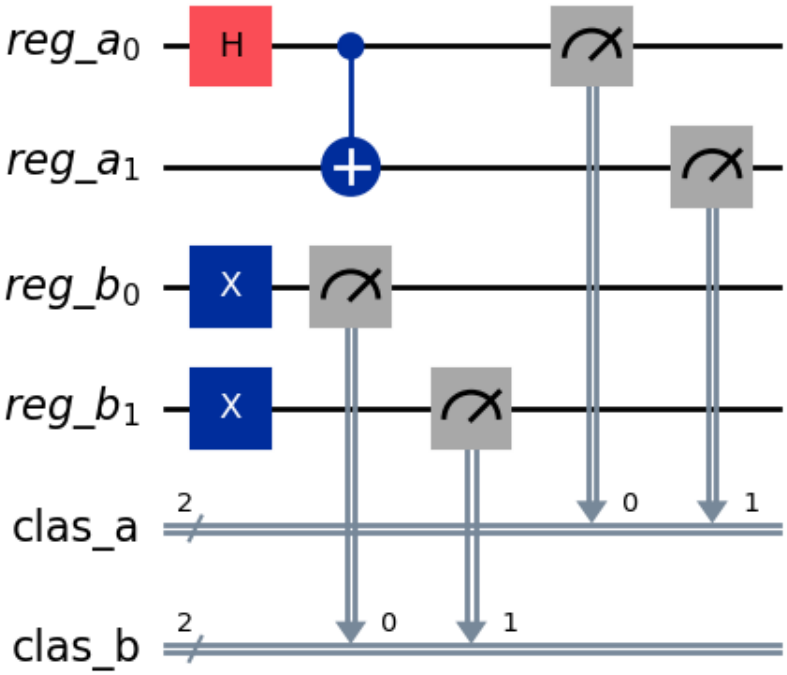
[5]:
```
circ.draw('mpl')
```

[5]:



[6]:
```
tr_circ0.draw('mpl')
```

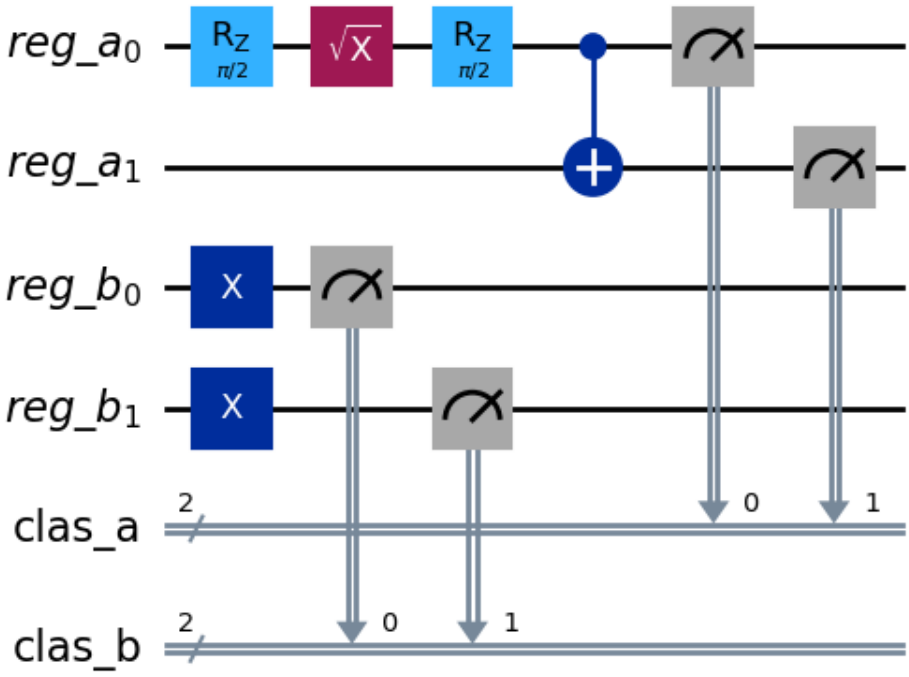
[6]:



[7]:
```
tr_circ1.draw('mpl')
```

[7]:

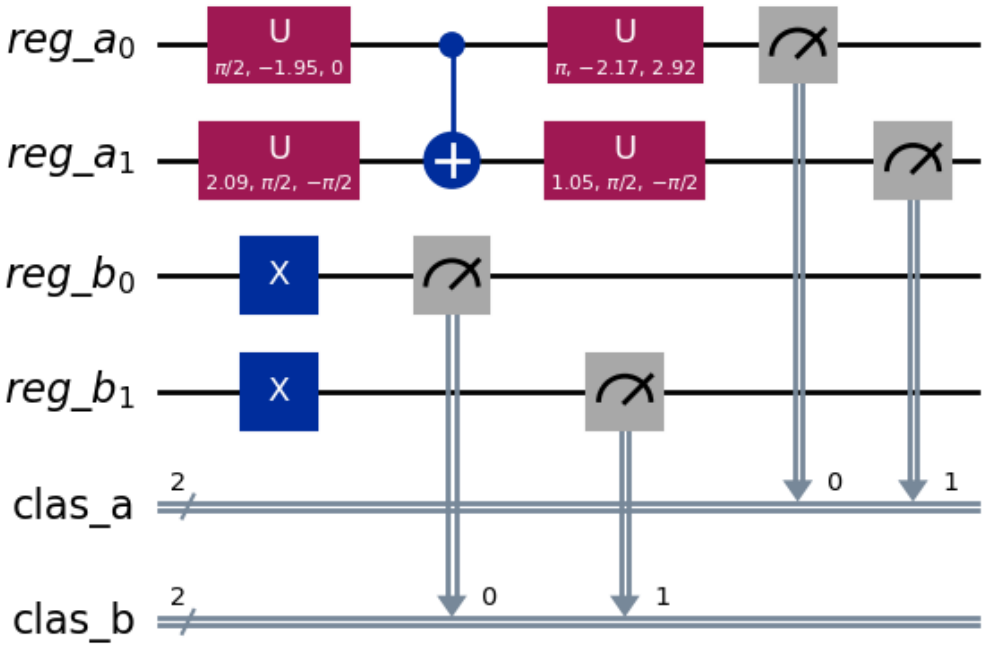
Global Phase: π/4



[8]:
```
tr_circ3.draw('mpl')
```

[8]:

Global Phase: 0.5967460233601163



[9]:
```
job=backend.run(tr_circ3, shots=1000,␣
 ↪job_tags=["BasicExample","Simulations","Student"])
print(job)
print()
print('Job id: ', job.job_id())
print('Program id: ', job.program_id)
print('Session id: ', job.session_id)
print('Tags: ', job.tags)
print('Usage estimation: ', job.usage_estimation)
```

```
<RuntimeJob('cnpe5l0almrcvvnl9clg', 'circuit-runner')>

Job id:  cnpe5l0almrcvvnl9clg
Program id:  circuit-runner
Session id:  None
Tags:  ['BasicExample', 'Simulations', 'Student']
Usage estimation:  {'quantum_seconds': None}
```

[10]:
```
res=job.result() # THIS INSTRUCTION WILL BLOCK THE PYTHON KERNEL UNTIL THE JOB␣
 ↪FINISHES!!
print(res)
```

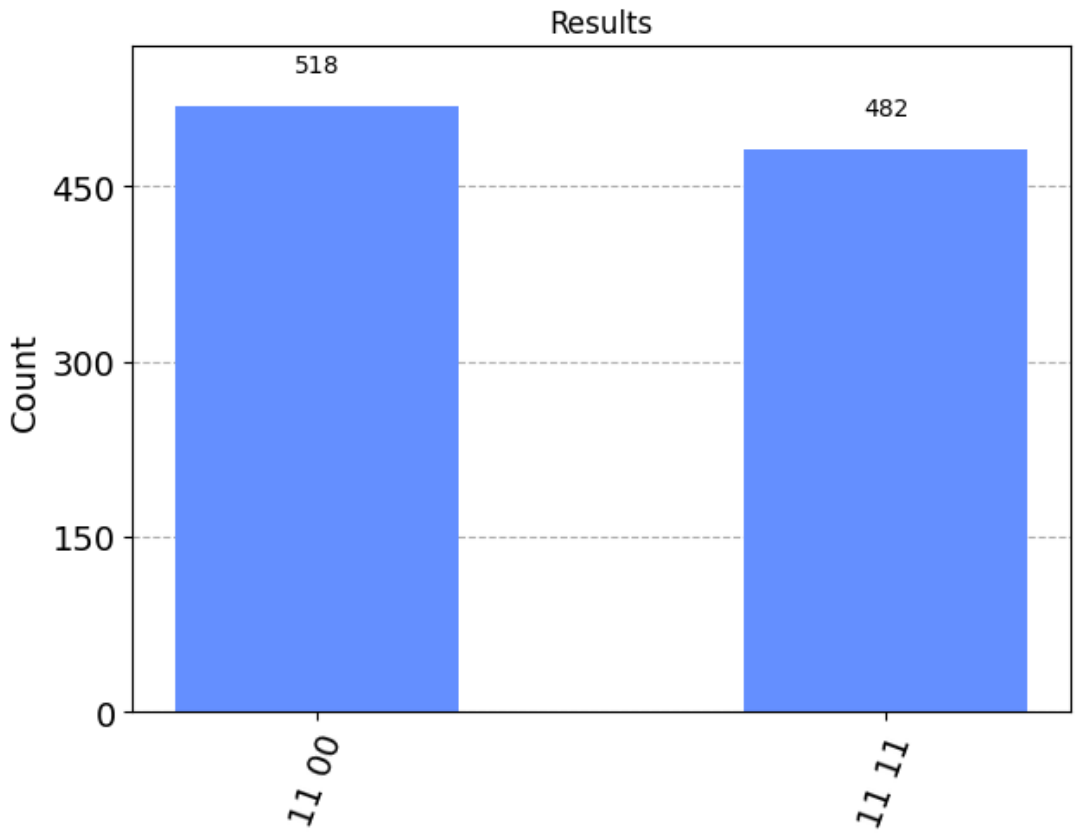
```
Result(backend_name='qasm_simulator', backend_version='0.13.3', qobj_id='',
job_id='Unknown', success=True, results=[ExperimentResult(shots=1000,
success=True, meas_level=2, data=ExperimentResultData(counts={'0xf': 482, '0xc':
518}), header=QobjExperimentHeader(creg_sizes=[['clas_a', 2], ['clas_b', 2]],
global_phase=0.5967460233601163, memory_slots=4, n_qubits=4, name='circuit-164',
qreg_sizes=[['reg_a', 2], ['reg_b', 2]], metadata={}), status=DONE,
seed_simulator=2050059534, metadata={'time_taken': 0.006114655,
'active_input_qubits': [0, 1, 2, 3], 'method': 'statevector', 'remapped_qubits':
False, 'num_clbits': 4, 'input_qubit_map': [[3, 3], [2, 2], [0, 0], [1, 1]],
'device': 'CPU', 'parallel_shots': 1, 'num_qubits': 4, 'measure_sampling': True,
'num_bind_params': 1, 'required_memory_mb': 1, 'batched_shots_optimization':
False, 'noise': 'ideal', 'sample_measure_time': 0.000913939, 'max_memory_mb':
64097, 'runtime_parameter_bind': False, 'parallel_state_update': 16, 'fusion':
{'applied': False, 'max_fused_qubits': 5, 'enabled': True, 'threshold': 14}},
time_taken=0.006114655)], date=2024-03-14T11:43:16.529114, status=COMPLETED,
header=None, metadata={'time_taken_execute': 0.00638326,
'time_taken_parameter_binding': 4.2884e-05, 'omp_enabled': True,
'max_gpu_memory_mb': 0, 'parallel_experiments': 1, 'max_memory_mb': 64097},
time_taken=0.008727073669433594)
```

[11]:
```
counts=res.get_counts()
print(counts)
```

```
{'11 11': 482, '11 00': 518}
```

[12]:
```
plot_histogram(counts, title='Results')
```

[12]:



[13]:
```
# STATISTICAL TEST

# Import SCIPY
import scipy as sc

# Test

test=sc.stats.binomtest(counts['11 00'],counts.shots(), 0.5)
print(test)
print()
print(test.pvalue)
```

```
BinomTestResult(k=518, n=1000, alternative='two-sided', statistic=0.518,
pvalue=0.2683726998591107)

0.2683726998591107
```